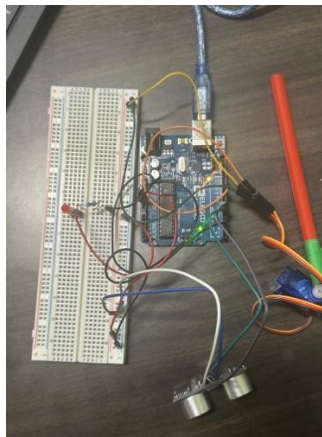


Numerical Methods (ECE3040-Project #2)

Heart Rate Prediction GUI

Group Members:

- Mina George
- Abid Ahmad
- Mustakim Choudhury



*This report was done with equal contribution from each member

Video: [YouTube Video](#)

Instructor: Abhilash Pandya

Date: 11/25/25

1. Introduction

This project focuses on a MATLAB system that measures the motion of a simulated heartbeat using an ultrasonic distance sensor and a servo motor, which replicates the position of the heartbeat. The implementation of a GUI was done to analyze and record the heartbeat. Furthermore, predictions were also done 100 milliseconds ahead within the GUI, which accounted for the delay of the servo. This project was conducted through the continuation of project 1, which integrated numerical methods, object-oriented programming, and the App designer to tie everything together. To further elaborate, the GUI collects distance versus time data from the surface that replicates the heartbeat, and displays it graphically with the predicted distances. This allowed the user to compute the oscillation frequency by locating zero crossings using Newton's divided differences. Least-squares regression and Lagrange polynomial interpolation were utilized as well to estimate

the future position of the moving surface. These predictions were visible in the GUI and could be directly sent to the servo motor to demonstrate the system delays.

2. System Design & Implementation

- ★ Arduino Uno Microcontroller
- ★ HC-SR04 ultrasonic sensor
- ★ LED indicator
- ★ Breadboard, resistor, and wires
- ★ Servo Motor

Connections: Trig – D7 | Echo – D6 | LED – D3 | VCC – 5V | GND – GND | SERVO-D11 (Supports Pulse Width Modulation (PWM))

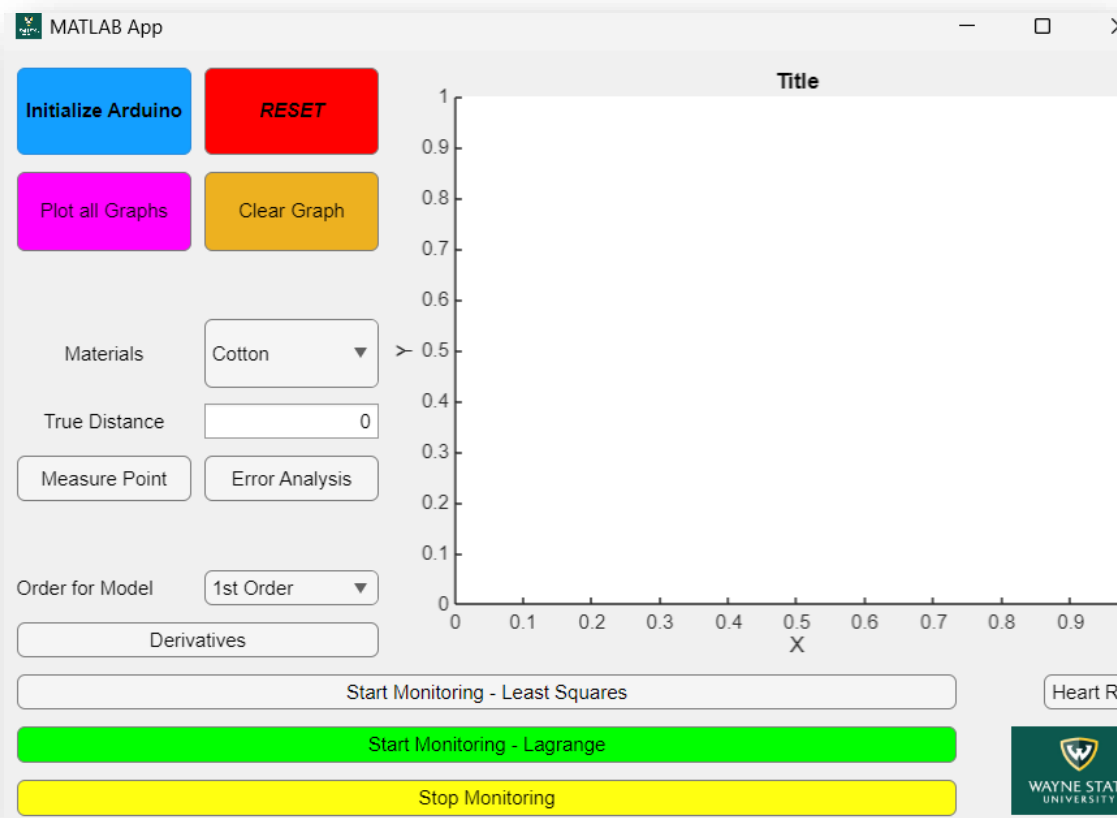


Figure 1: As shown in the figure above, the GUI interface initializes all Arduino hardware, collects live distance data, and lets the user select between the least squares and Lagrange prediction, computes frequencies with user-selected points, and plots the graph.

Code Snippet- InitializeArduinoButtonPushed

```
function InitializeArduinoButtonPushed(app, event)
%Mina
%Our pins for the Arduino Board, we can update if needed
app.port = 'COM3';
app.board = 'Uno';

app.arduino = arduino(app.port, app.board, 'Libraries', {'Servo', 'Ultrasonic'}); %INCLUDE BOTH SERVO AND SENSOR LIBRARY SO THEY CAN BE USED

trig = 'D7'; %pin 7
echo = 'D6'; %pin 6
led = 'D3'; %pin 3
app.sensor = DistanceSensor(app.arduino, trig, echo, led); %initialize our sensor

%initializing the servo motor
servoPin = 'D11'; %pin 11 has the tilda, it supports PWM (REQUIRED FOR THE SERVO MOTOR)
app.servo = Motor(app.arduino, servoPin); %initialize our servo

disp('Servo and sensor initialized succesfully'); %display in our command window so that we ensure everything is initialized
end
```

Code Snippet- Initializing GUI

```
% Create UIFigure and components
function createComponents(app)

% Get the file path for locating images
pathToMLAPP = fileparts(mfilename('fullpath'));

% Create UIFigure and hide until all components are created
app.UIFigure = uifigure('Visible', 'off');
app.UIFigure.Position = [100 100 709 480];
app.UIFigure.Name = 'MATLAB App';
app.UIFigure.Icon = fullfile(pathToMLAPP, 'wsu-primary-stacked-reversed300.jpg');
```

3. Problem Description

Different environments demand different real-time systems. For example, medical robots, or small monitors that interact with moving surfaces. As a result, the controller or the systems must react and work with the motion that is already changing by the time the command is sent. For this project, we modeled a situation where we use an ultrasonic sensor and a servo motor whose pointer is attached to its arm. The ultrasonic sensor measures the distance to a moving surface object that stimulates either a beating heart, breathing lungs, or a similar object. Given that data acquisition and computation take a finite amount of time, as well as the servo delay (on the order of 100 ms), the system will not be able to accurately respond to the current distance. By the time the system reacts, the surface has already moved to a different position. As a result, the engineering problem for this project is to estimate the future position of a surface and make the system and servo keep up with the motion as accurately as possible.

To address this engineering problem, we needed to collect distance versus time data that uses numerical methods to compute the heart breathing rate and predict future displacement of the surface. The system will be able to compute the frequency of motion by finding zeros of the

distance signal using the user's selected points of the waveform. After that, the system will use a small window of the recent data that was collected to build mathematical models of the motion and predict the distance 100ms ahead of time. Furthermore, the system will use both the least squares method (linear, quadratic, and cubic) and the Lagrange polynomial interpolation to predict future distance, and the system will compare the data and display it on the screen. After the system calculates the future displacement, it is mapped to the servo angle so the point will be able to follow the moving surface in real time with maximum accuracy. All these features are integrated into the MATLAB AppDesigner GUI, which can be operated by the user for data collection, visualization, and system control.

4. Numerical Solutions

In the AppDesigner GUI, the system will start by collecting the distance and time data from the ultrasonic sensor at a sampling interval of about 0.1s; the system stores this data in vectors t and d . To calculate the heart rate or the moving object rate, the signal is centered around zero by subtracting its mean, so that the data oscillates around zero. After that, the user can use the automatic pointer to click two points that will be utilized to compute the frequency. The computation is based on the assumption that the user selected two points which are approximately one period apart. From the selected data points, the system will search forward to find a sign change around the centered data, and will bracket the zero crossing. Once this is completed, the system will utilize the Newton-Divided difference function (linear interpolation between the bracketed samples) so that it can estimate the correct zero-crossing times. This data is then used to calculate the period, frequency, and to provide a rate estimate of the noisy signal. This system can be configured such that it will estimate heart rate or moving object rate from data for a long period of time.

As mentioned above, the system employs both least squares of polynomial regression and Lagrange polynomial interpolation. In each real-time data collection, the system will take the most recent order+1 samples, and it will either fit a polynomial degree 1-3 (least squares), or it will build a Lagrange polynomial interpolation using those points. The model will then be evaluated at future times, which is represented by $t_future = t_current + 0.1$, to predict the distance about 100 ms ahead of time, attempting to account for the servo delay. The predicted distance will then be converted to a servo angle using linear scaling. Running this system will plot the measured and predicted distances in the GUI, and the plots use complete numerical solutions including zero-finding, regression, and interpolation to compute motion and to drive the servo motor accurately.

Code Snippet- Plotting True vs Measured Data

```
function MeasurePointButtonPushed(app, event)
%MINA
selectedMaterial = app.MaterialsDropDown.Value;           %get selected material from the drop down menu

trueValue = app.TrueDistanceEditField.Value;             % Get true distance (user input from ruler)

measuredValue = app.sensor.read();                       % Read measured value from ultrasonic sensor

app.trueValues(end+1) = trueValue;
app.measuredValues(end+1) = measuredValue;

app.simulationCount = app.simulationCount + 1;           %increase simulation count
app.simulationData{app.simulationCount} = struct('material', selectedMaterial, 'true', trueValue, 'measured', measuredValue);

% Update and plot data
cla(app.UIAxes);
hold(app.UIAxes, 'on');                                  %allows multiple points to be graphed at once
for k = 1:app.simulationCount
    appData = app.simulationData{k};
    plot(app.UIAxes, appData.true, appData.measured, 'o-'); %plot the true values against the measured values
end
hold(app.UIAxes, 'off');

%title and labels
xlabel(app.UIAxes, 'True Distance (m)');
ylabel(app.UIAxes, 'Measured Distance (m)');
title(app.UIAxes, 'True vs Measured Distance');
end
```

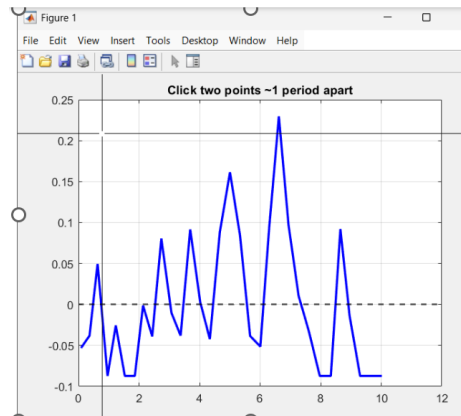


Figure 2: As shown in the figure above, the user selects two points roughly one period apart on the waveform. Then the system finds the forward zero crossing and subsequently computes the frequency

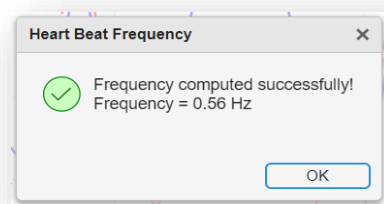


Figure 3: GUI popup screen showing the computed frequency from two user-selected data points

5. Discussion

Wayne State University

College of Engineering — Department of Electrical and Computer Engineering

This project demonstrated that both the least squares method and Lagrange interpolation can be utilized to predict the future motion of the moving surface; however, they behave a bit differently in real life. During the testing phase, it was discovered that the low-order least-squares fits, like 1st and 2nd order, were used to smooth out noise better and gave more reliable predictions than Lagrange interpolation; however, Lagrange passes exactly through the data points, which also makes the data very accurate. The Lagrange method seems to do better with small measurement errors, but it is a bit more sensitive to noise. Using a higher-order polynomial or using more points can sometimes improve the fit, but it could also introduce more oscillations and more errors when doing a prediction for 100ms in the future.

Both methods produced accurate results, and both methods went through robust testing. The prediction of only one step ahead (about 0.1s) kept the pointer of the motor in alignment with the moving object. If values are predicted further than 100 ms, the pointer may not follow the movement as accurately. In addition, predicting further than 100ms will introduce errors in the calculation. The servo has a limited range and speed, so even a good prediction cannot be replicated properly if the motion is too fast or if the distance changes too suddenly. In terms of collection, both Lagrange and least squares were fast and accurate enough to calculate in 0.1 loops. Going above 0.1s would require more accurate techniques like the Newton-Euler method. The main limitations of this system will come from sensor noise, sampling rate, and servo limitations rather than MATLAB's computational capabilities. The observation also highlights that real-time tracking systems, such as the heartbeat rate sensor, are better suited to use a lower-order regression model with a short prediction distance because they are more reliable than higher-order interpolation models.

Code Snippet- Lagrange

```
function [d_pred, theta_deg] = lagrange(app, t, d, order)
    L = 0.1; % Servo arm length (meters)

    % Ensure numeric arrays
    t = double(t);
    d = double(d);
    n = length(t);

    if n <= order
        t_points = t;
        d_points = d;
    else
        t_points = t(end-order:end);
        d_points = d(end-order:end);
    end

    % Predict at future time, 100ms into future
    t_future = t(end) + 0.1;

    % Lagrange interpolation formula
    d_pred = 0;
    nPoints = length(t_points);
    for i = 1:nPoints
        L_i = 1;
        for j = 1:nPoints
            if i ~= j
                L_i = L_i * (t_future - t_points(j)) / (t_points(i) - t_points(j));
            end
        end
        d_pred = d_pred + d_points(i) * L_i;
    end

    % Clamp predicted distance
    d_pred = max(min(d_pred, L), -L);

    % Compute servo angle
    theta_deg = asin(d_pred / L) * (180/pi);
    theta_deg = max(0, min(180, theta_deg));
end
```

Wayne State University
 College of Engineering — Department of Electrical and Computer Engineering
Code Snippet-Least_Squares

```
function [d_pred, theta_deg] = least_squares(app, t, d, order)
    L = 0.1; % Servo arm length, 10cm

    % Use only the last (order+1) points
    n = length(t);
    if n <= order
        t_used = t;
        d_used = d;
    else
        t_used = t(end-order:end);
        d_used = d(end-order:end);
    end

    % Fit polynomial and predict future distance using polyfit and
    % polyval (least squares)
    p = polyfit(t_used(:), d_used(:), min(order, length(t_used)-1));
    t_future = t(end) + 0.1; %predict 100ms into the future to accomodate for delay
    d_pred = polyval(p, t_future); %get distance prediction based off our regression model and future time

    % Clamp predicted distance to avoid asin error
    d_pred = max(min(d_pred, L), -L);

    % Compute servo angle
    theta_deg = asin(d_pred / L) * (180/pi);
    theta_deg = max(0, min(180, theta_deg)); % Clamp angle
end
```

6. Sample Prediction Results

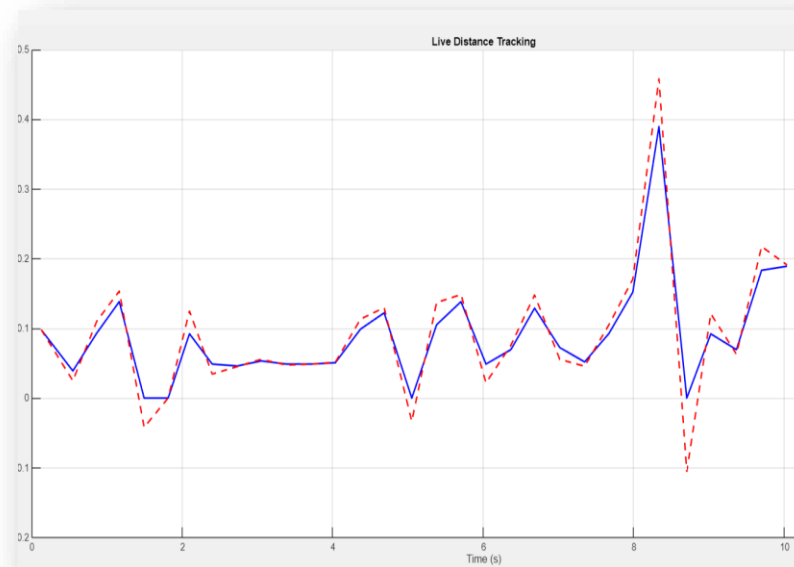


Figure 4: Measured distance and linear (1st-order) least squares prediction vs. Time.

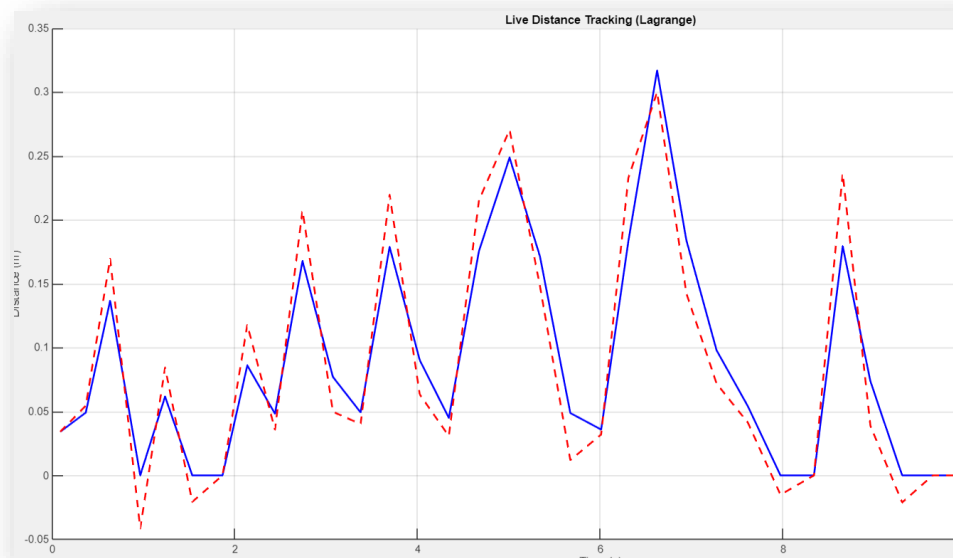


Figure 5: Measured distance and Lagrange Interpolation prediction vs. Time

7. Conclusion

After the completion of this project, our team was able to successfully combine numerical methods, real-time sensing, and mechatronic actuation into a system built from object-oriented programming, which was capable of analyzing and following a simulated heartbeat motion. The system computed the oscillation frequency by locating the zero crossings with Newton's divided difference numerical method. This demonstrated how numerical techniques can extract meaningful physical parameters from sensor data. Motion tracking delay was reduced by using least squares regression and Lagrange polynomial interpolation to estimate the distance 100 milliseconds ahead, attempting to account for the delay of the servo motor. The use of these numerical methods within the project highlighted each of their strengths and weaknesses. For instance, least squares was good at isolating noise, whereas Lagrange interpolation modeled short-term movement more accurately. The demonstration of the servo being able to follow either the real-time data or the predicted data showed how prediction can improve the ability of a mechanical actuator to keep up with an irregular surface.

Wayne State University

College of Engineering — Department of Electrical and Computer Engineering